

SAE 2.02 Comparaison d'approches algorithmiques Exploration de donjons

SUJET DU PROJET

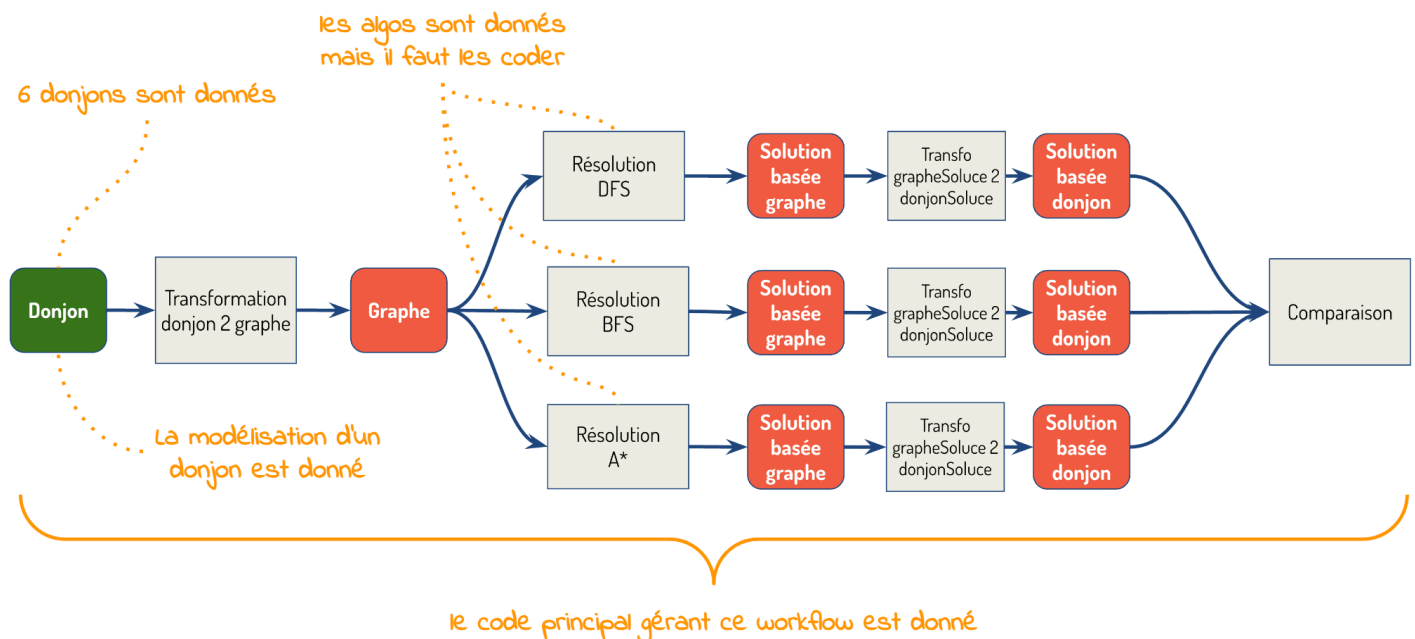
Il s'agit de trouver le chemin le plus court entre 2 salles d'un donjon.

Pour cela, la résolution devra être réalisée sur une version "graphe" correspondant au donjon, et non directement au travers des concepts spécifiques du donjon. Cette approche consiste à transposer un problème dans un contexte où ce type de problème sait être traité. De même, la solution au niveau graphe devra être transposée pour correspondre à une solution au niveau du donjon.

Au niveau graphe, vous aurez 3 algorithmes de résolution différents à implémenter et comparer : **Deep-First-Search**, **Breadth-First-Search** et **A***.

ASPECTS TECHNIQUES DU PROJET

Le schéma suivant illustre l'ensemble des étapes demandées (les boîtes rectangulaires).



Notez que de nombreux éléments vous seront donnés.

FONCTIONNALITÉS DEMANDÉES

1. Code de transformation du donjon en graphe
2. Code de résolution du problème sur le graphe avec DFS
3. Code de résolution du problème sur le graphe avec BFS
4. Code de résolution du problème sur le graphe avec A*
5. Code de transformation de la solution du graphe (liste de noeuds) en solution pour le donjon (liste de directions)
6. Ajout de 2 donjons (via le format fichier texte)
7. Analyse des résultats obtenus pour les 6 (donnés) + 2 (ajoutés) donjons.



ASPECTS TECHNIQUES POUR LA RÉALISATION DU PROJET

Vous devez écrire un programme **JAVA** (IDE Eclipse) qui s'exécutera en mode console.
Attention à ne pas coder avec une version JAVA trop élevée (8 max - idéalement 7)

Tout le code donné est disponible à ce dépôt : [LIEN](#)

Afin de vous aider si c'est nécessaire, nous vous proposons des diagrammes de classes (en annexe) de la solution que nous avons implémentée. Ces diagrammes ne montrent pas forcément les attributs.

COMPARAISON DES APPROCHES ET MESURES DE PERFORMANCES

L'objectif du projet est également de comparer les performances des différents algorithmes selon les donjons.

Étant donné la rapidité d'exécution, nous vous proposons d'observer le temps pour 10000 résolutions consécutives. Vous veillerez à faire vos relevés sur la même machine.

Un tableau sera construit avec ces informations :

	DFS			BFS			A*		
Donjon	Solution	Steps	Temps	Solution	Steps	Temps	Solution	Steps	Temps
1									
2									
...									
8									

Pour chaque donjon vous devrez analyser et expliquer les résultats obtenus afin de prouver votre bonne compréhension des différents algorithmes.

La solution sera exprimée à niveau donjon (exemple [EAST, NORTH, SOUTH, WEST]).

LIVRABLES

Vous aurez à produire les livrables suivants :

- l'application complète (sera rendue sous forme de **ZIP** contenant le **projet Eclipse complet**)
- un rapport (format **PDF**) contenant le tableau complété et la répartition du travail dans le binôme
 - ce qui a été fait versus pas fait ;
 - les donjons ajoutés
 - le tableau de comparaison...
 - qui a fait quoi ;
 - % de travail pour chacun.

Ces 2 livrables (ZIP+PDF) seront rendus en un seul **ZIP** général sur **UMtice** par **un seul** des 2 membres du groupe.

Date limite de remise : Vendredi 10 avril 2026 à 20h00



ÉVALUATION

Elle sera basée sur les critères suivants :

- fonctionnalités implémentées
- qualité des résultats obtenus et de leur analyse
- qualité du code Java
- qualité de la POO

Nous nous gardons le droit de donner des notes différentes par membre d'un groupe si l'investissement est significativement déséquilibré entre eux.

INFORMATIONS ADDITIONNELLES

Code de transformation du donjon en graphe

- la transformation est réalisée lors de l'appel au constructeur **Dungeon2Graph**
- la méthode **Node mappedNode(Room room)** ne prend qu'une ligne : elle indique quel noeud correspond à la room dans le graphe qui a été construit
- l'utilisation de dictionnaires est très utile dans la classe **Dungeon2Graph** afin de mémoriser l'association *Room*-> *Node*
- Les coordonnées des Room sont transmises au noeud correspondant
 - elles ne sont utiles QUE pour le calcul de distance (heuristique) pour le A*
 - ne les utilisez pas pour les transformations car le système de coordonnées des donjons 1&2 n'est pas le même que celui des donjons 3 à 6 (inversion axe ordonnée).
- Dictionnaire en Java : Map / HashMap
 - ajout : put(clé, valeur)
 - récupérer la valeur pour une clé => get(clé)
 - possibilité d'itérer
 - sur les clés => keySet()
 - sur les valeurs => values()
 - sur les couples clé/valeur => entrySet()

Code de résolution du problème sur le graphe avec DFS

- positionnez bien votre compteur de **steps** dans le maximum de boucles pour qu'il reflète bien le nombre de "passes" de l'algo.

Code de résolution du problème sur le graphe avec BFS

- positionnez bien votre compteur de **steps** dans le maximum de boucles pour qu'il reflète bien le nombre de "passes" de l'algo.

Code de résolution du problème sur le graphe avec A*

- idem BFS
- la file prioritaire peut être réalisée à partir de Java 7 avec **PriorityQueue**
- cette classe propose un constructeur prenant en paramètre un **Comparator**
 - à vous de lui fournir donc un objet compatible avec cette interface
 - dans votre cas il s'agira d'un **Comparator<Node>**
 - vous aurez donc à définir le corps d'une méthode **public int compare(Node s1, Node s2)** qui renverra -1, 0 ou 1 selon

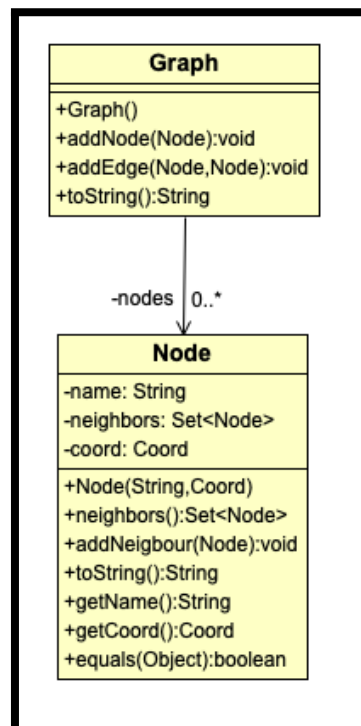
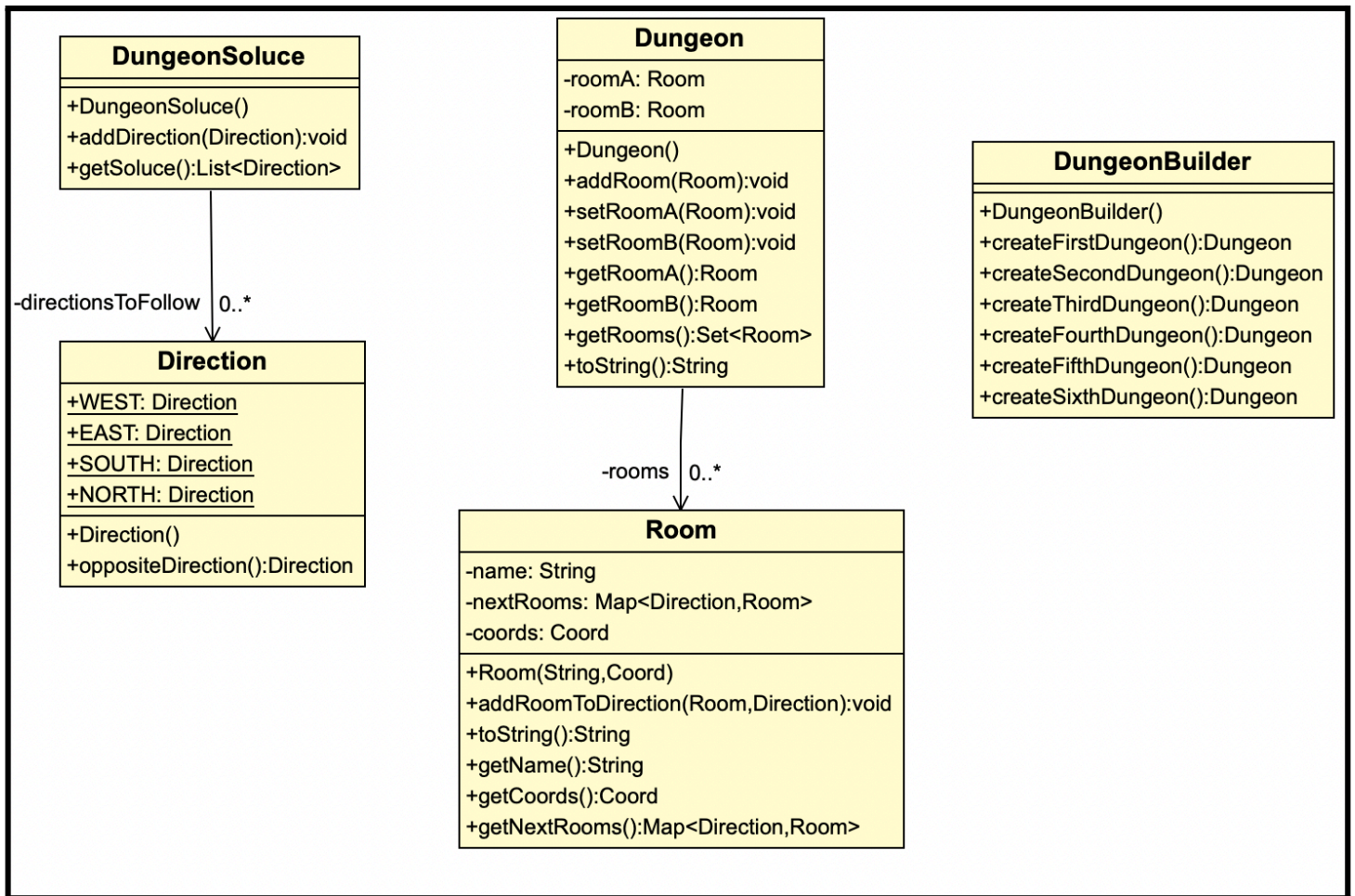


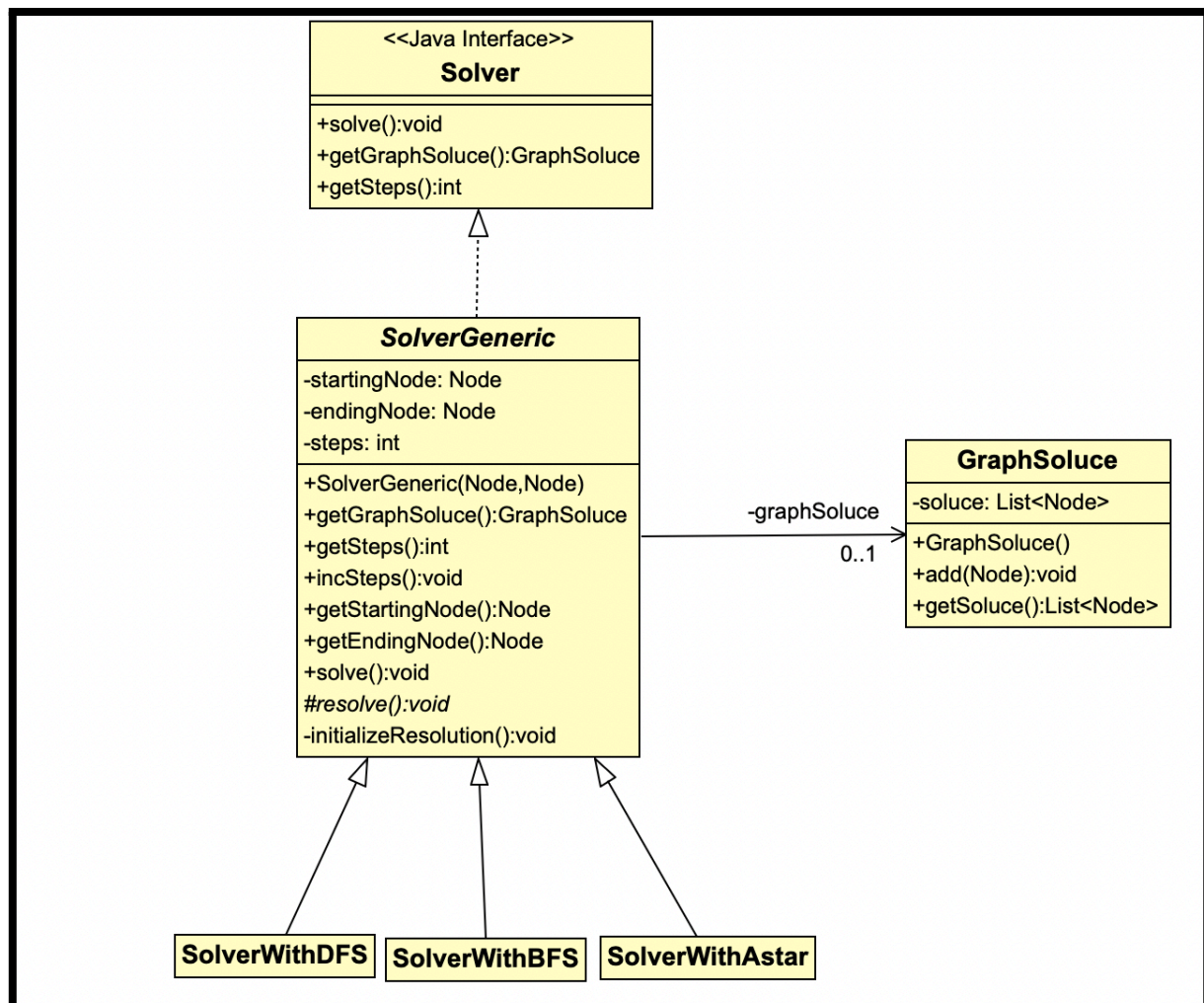
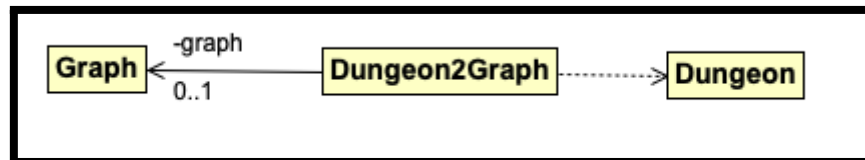
Code de transformation de la solution du graphe en solution pour le donjon

- la transformation est prise en charge par ***DungeonSolu******ce transform(GraphSolu******ce solu******ceGraphBFS)***
- cette méthode est appelée sur votre instance ***mapping*** créée lors de la traduction donjon vers graphe
 - elle peut ainsi profiter d'une éventuelle mémorisation Node -> Room que vous auriez eu la bonne idée de conserver en construisant initialement le graphe.



ANNEXES





Algorithme 1 Algorithme de parcours en profondeur d'abord d'un graphe g . À la fin de l'algorithme $\pi(s)$ est le prédécesseur de s sur le parcours obtenu.

Fonction DFS(g : Graphe)

```
1: pour chaque nœud  $s$  de  $g$  faire
2:   si  $s$  n'est pas marqué alors
3:      $\pi(s) \leftarrow null$ 
4:     Prof( $G, s$ )
5:   fin si
6: fin pour
```

Fonction Prof(g : Graphe, i : Nœud)

```
1: marquer le nœud  $i$ 
2: pour chaque voisin  $v$  de  $i$  faire
3:   si  $v$  n'est pas marqué alors
4:      $\pi(v) \leftarrow i$ 
5:     Prof( $G, v$ )
6:   fin si
7: fin pour
```

Algorithme 2 Algorithme de parcours en largeur d'abord d'un graphe g . Cet algorithme utilise une file comme structure de données (FIFO). A la fin de l'algorithme $\pi(s)$ est le prédécesseur de s sur le parcours obtenu.

Fonction BFS(g : Graphe, depart : Nœud)

```
1: file = File()
2: marquer depart
3: file.ajouter(depart)
4: tant que file n'est pas vide faire
5:    $s = \text{file.defiler}()$ 
6:   pour chaque voisin  $v$  de  $s$  dans  $g$  faire
7:     si  $v$  n'est pas marqué alors
8:        $\pi(v) = s$ 
9:       marquer  $v$ 
10:      file.ajouter( $v$ )
11:    fin si
12:  fin pour
13: fin tant que
```

REMARQUE : la reconstitution de la solution (en remontant les noeux prédécesseurs) n'est pas visible dans les 2 algorithmes ci-dessus mais fait pourtant parti de l'implémentation de la résolution (i.e. à faire **après** le tant que).



Algorithme 3 Algorithme A* pour la recherche du plus court chemin dans un graphe non valué. Cet algorithme utilise une file et une file prioritaire comme structures de données.

Fonction cheminPlusCourt(g : Graphe, objectif : Nœud, depart : Nœud)

```
1: closedList = File()
2: openList = FilePrioritaire(comparateur = compareParHeuristique)
3: openList.ajouter(depart)
4: tant que openList n'est pas vide faire
5:   u = openList.defiler()
6:   si u.x == objectif.x et u.y == objectif.y alors
7:     reconstituerChemin(u)
8:     terminer le programme
9:   fin si
10:  pour chaque voisin v de u dans g faire
11:    si non (v existe dans closedList ou v existe dans openList avec un coût inférieur)
12:      alors
13:        v.cout = u.cout + 1
14:        v.heuristique = v.cout + distance([v.x, v.y], [objectif.x, objectif.y])
15:        openList.ajouter(v)
16:    fin si
17:  fin pour
18:  closedList.ajouter(u)
19: fin tant que
```
